

Kryptographie und Embedded-Security

IoT Workshop

Fredy Hirt - 02.09.2026

Previously on

- Ihr versteht die grundlegenden Unterschiede zwischen Authentifizierung und Autorisierung
- Ihr kennt die Mechanismen zur Authentifizierung und Autorisierung in MQTT
- Ihr unterscheidet zwischen physischer und logischer Topic-Modellierung

Agenda

- Repetition Authentifizierung & Autorisierung (15')
- Input Kryptographie (40')
- Input Embedded-Security (35')
- Arbeit am Projekt (90')

Ziele

- Ihr versteht die grundlegende Funktionsweise von
 - symmetrischer Kryptographie
 - asymmetrischer Kryptographie
- Ihr kennt die grundlegenden Herausforderungen an Security auf Embedded-Systems

Motivation

- Wer hat einen Staubsaugroboter?

Motivation

👉 Lies den Artikel [The DJI Romo robovac had security so poor, this man remotely accessed thousands of them](#) durch

Symmetrische Kryptographie

- Ein gemeinsamer geheimer Key
- Sender und Empfänger kennen denselben Key

Klartext + Key $\rightarrow$ Chiffretext

Chiffretext + Key $\rightarrow$ Klartext

Eigenschaften

- ✓ Schnell
- ✓ Geringer Rechen- und Speicheraufwand
- ✓ Gut für grosse Datenmengen

Eigenschaften

- ✗ Keys müssen sicher verteilt werden
- ✗ Keys müssen sicher gespeichert werden
- ✗ Ein kompromittierter Key kompromittiert alle
Geräte die denselben Key verwenden
- 💡 Ein Key pro Gerät begrenzt den Schaden bei einer
Kompromittierung

Typische symmetrische Algorithmen

- AES-128 / AES-256
- ChaCha20

Typische sichere Kombinationen:

- AES-GCM
- ChaCha20-Poly1305

Asymmetrische Kryptographie

- Zwei zusammengehörige Schlüssel:
 - **Public Key**
 - **Private Key**
- **Public Key** darf verteilt werden
- **Private Key** muss geheim bleiben

Asymmetrische Kryptographie

Verschlüsselung

Public Key → verschlüsseln

Private Key → entschlüsseln

Digitale Signatur

Private Key → signieren

Public Key → Signatur prüfen

Eigenschaften

- ✓ Kein Shared Secret nötig
- ✓ Digitale Signaturen für Authentizität

Eigenschaften

- ✗ Komplexer als symmetrische Kryptographie
- ✗ Höherer Rechen- und Speicheraufwand
- ✗ Private Keys müssen besonders geschützt werden

Typische asymmetrische Algorithmen

- RSA (Signaturen & Verschlüsselung)
- ECDSA (Signaturen)
- ECDHE (Key Exchange)

Welches Verfahren passt?

- Prüfen, ob Firmware tatsächlich vom Hersteller stammt?
- Effiziente Übertragung von Sensordaten?
- Gemeinsamen Sitzungsschlüssel zwischen Client und Broker aushandeln?

Welches Verfahren passt?

- Firmware vom Hersteller → Digitale Signatur (z.B. ECDSA)
- Sensordaten übertragen → Symmetrisch (z.B. AES-GCM)
- Sitzungsschlüssel aushandeln → Asymmetrisch (z.B. ECDHE)

Kryptographie in MQTT

☞ MQTT selbst enthält keine Kryptographie
Security wird durch **TLS** bereitgestellt.

MQTT + TLS = MQTTS

Was macht TLS?

- TLS kombiniert:
 - Asymmetrische Kryptographie
 - Symmetrische Kryptographie

Warum Kombination?

- Asymmetrisch:
 - Broker authentifizieren
 - Clients authentifizieren
 - Session-Key aushandeln
- Symmetrisch:
 - MQTT-Daten effizient verschlüsseln
 - Integrität der Nachrichten schützen

PKI – Public Key Infrastructure

Problem:

Woher weiss der Client, dass der Broker echt ist?

Lösung

Zertifikate enthalten:

- **Public Key**
- Identität
- Signatur einer **CA** (Certificate Authority)

Lösung (Chain of Trust)

Root CA



Intermediate CA



Server-Zertifikat

👉 Wenn Root vertraut wird, wird der gesamten Kette vertraut.

Embedded-Security

- Geräte sind physisch zugänglich
- Speicher (Flash) kann ausgelesen werden
- Firmware kann manipuliert werden
- Keys können extrahiert werden
- Side-Channel-Angriffe
- Fault Injection
- Debug-Interfaces (JTAG)

Secure Boot

Nur vertrauenswürdig signierte Software wird gestartet.

- Ist die Firmware digital signiert?
 - Ja → Start
 - Nein → Stop
- Schützt vor manipulierten Firmware-Images

Was Secure Boot nicht schützt

- das Auslesen der Firmware
- das Extrahieren von Schlüsseln

Dafür braucht es zusätzliche Massnahmen:

- Debug-Interfaces (JTAG) sperren (fusen)
- Flash-Auslesen verhindern
- Schlüssel in geschütztem Speicher ablegen

Firmware Signing

- Hersteller:
 - Signiert Firmware mit **private Key**
- Gerät:
 - Prüft Signatur mit **public Key**

OTA-Updates

Over-The-Air Updates müssen:

- Signiert sein
- Versioniert sein
- Verschlüsselt übertragen werden
- Rollback-Schutz besitzen

Weiterführendes

- IoT Security Breaches: 4 Real-World Examples
- Sicherheitslücke im LTE-Netz